# Cache Tier Manager

In database caching, managing data efficiently between fast RAM and slow disk storage is critical for system performance. A two-tier storage system classifies cached data as either **HOT** (frequently accessed, fast retrieval) or **COLD** (infrequently accessed, slower retrieval) to optimize resource usage and query latency.

Your task is to implement a class `TopKCache` that manages data movement between two storage tiers. The system keeps all data but classifies keys into **HOT** (top K most recently used) or **COLD** (all other stored keys) based on usage recency. The cache must support insertions, retrievals, and status queries while maintaining correct tier classification.

### Class Methods

- `__init__(self, k)`: Initializes the cache with a **HOT** capacity of `K` key-value pairs.
- `put(self, key, value)`: Stores the key-value pair in the cache and marks it as the most recently used.
- `get(self, key)`: Returns the value corresponding to `key` and marks it as the most recently used. Returns `None` if the key is not present.
- `get_status(self, key)`: Returns `HOT` if the key is in the top K recently used keys, `COLD` if it exists but is not in the top K, or `None` if missing. This method **does not modify the recency order**.

### Example

Input:

```
8
init 2
put A 10
put B 20
put C 30
status A
get A
status A
status B
```

Output:

```
COLD
10
HOT
COLD
```

Explanation:

- init 2: Top 2 items are HOT.
- put A, put B: The order is [B, A]. Both are HOT.
- put C: Order is [C, B, A]. The recent 2 keys, C and B are HOT, while A becomes COLD.
- status A: Returns the string "COLD".
- get A: A is accessed. The order becomes [A, C, B]. The recent 2 keys, A and C are HOT, while B becomes COLD.
- status A: Returns the string "HOT".

- status B: Returns the string "COLD".

Note:

### Constraints

- 1 <= Number of operations (Q) <= 10^5.
- 1 <= K <= 1000
- keys and values are alphanumeric strings.

### Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the class methods:

The first line contains an integer Q, the total number of commands. Each of the next Q lines contains a command in one of the following formats:

- `init <k>`: Initializes TopKCache with capacity k
- `put <key> <value>`: Calls `put(key, value)`
- `get <key>`: Calls `get(key)` and prints the result
- `status <key>`: Calls `get_status(key)` and prints the result